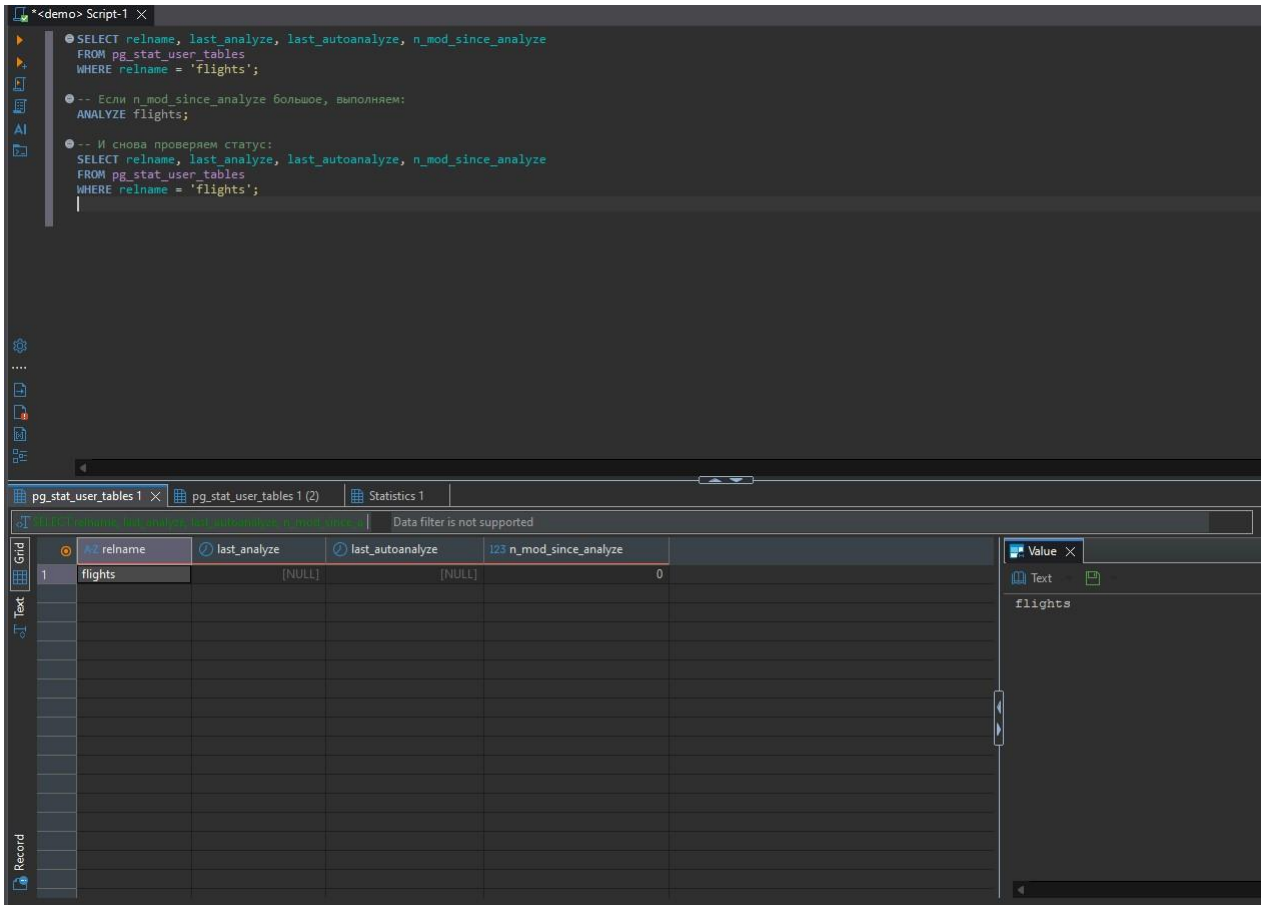
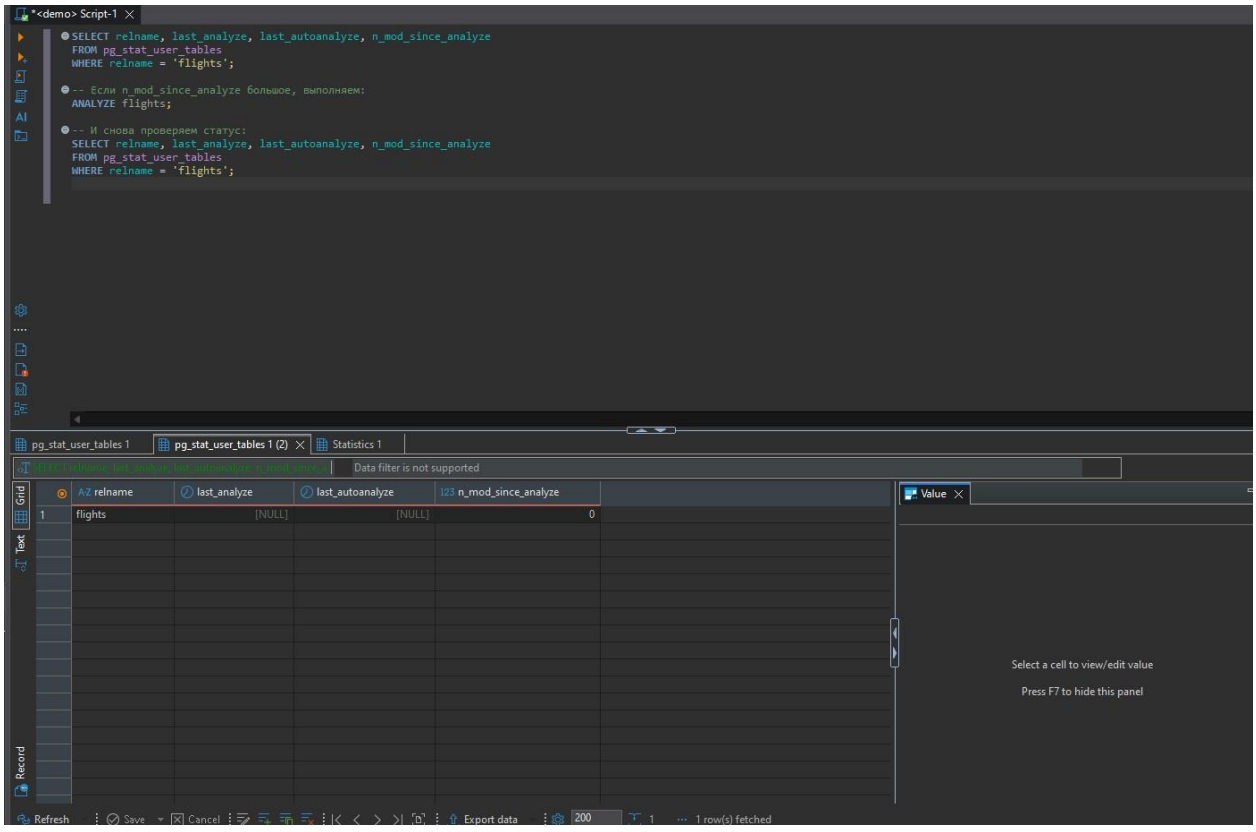
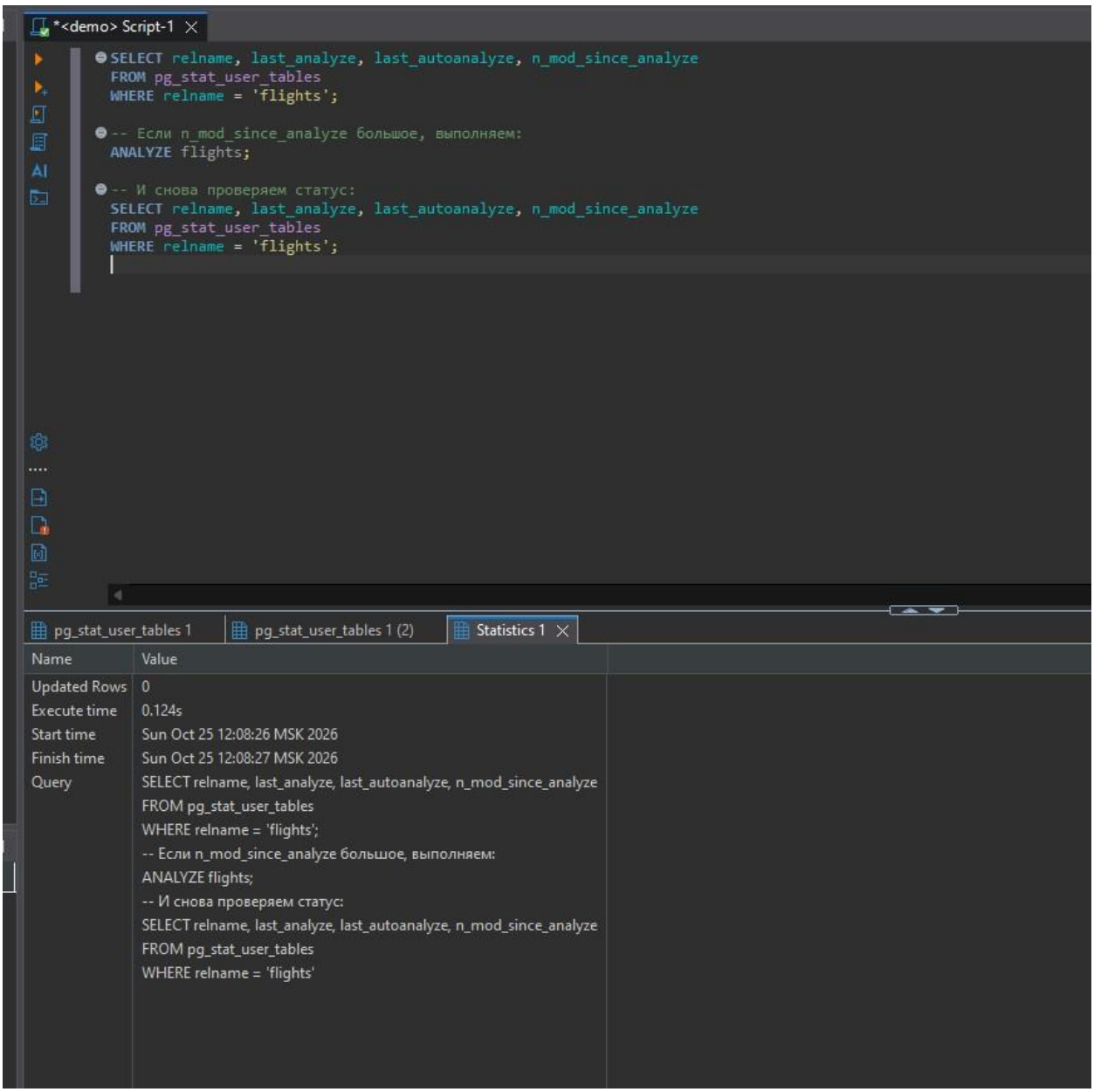
1. Pre-flight Checklis







В первую очередь я проверила актуальность статистики для таблицы flights. Для этого я обратилась к системному представлению pg_stat_user_tables

Я определяю, что статистика устарела, по колонке n_mod_since_analyze - она показывает, сколько строк было изменено, добавлено или удалено с момента последнего сбора статистики. Если это значение превышает 10% от общего объема таблицы, значит, пора её обновлять

Как видно на моих скриншотах, у меня значение n_mod_since_analyze оказалось равно 0, а даты в колонках анализа стояли как [NULL]. Это означает, что данные в таблице не менялись, и обновлять статистику по условию задания мне не требовалось. Если бы я всё же выполнила ANALYZE flights, то счетчик n_mod_since_analyze просто сбросился бы на ноль, а в last_analyze зафиксировалось бы текущее время

2. Анализ кэша и прогрев

```
-- 1. Фиксация холодного кэша
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
  AND f.scheduled_departure < '2017-09-01';

-- 2. Прогрев таблиц
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
SELECT pg_prewarm('ticket_flights');
```

Results 1 | Results 1 (2) | Results 1 (3) | Statistics 1

Data filter is not supported

	QUERY PLAN
1	Hash Join (cost=2025.79..51545.41 rows=600508 width=31) (actual time=6.981..361.359 rows=678171 loops=1)
2	Hash Cond: (tf.flight_id = f.flight_id)
3	Buffers: shared hit=15367 read=5185
4	-> Seq Scan on ticket_flights tf (cost=0.00..43323.35 rows=2360335 width=24) (actual time=0.007..111.503 rows=2360335 loops=1)
5	Buffers: shared hit=15335 read=4385
6	-> Hash (cost=1816.96..1816.96 rows=16706 width=11) (actual time=6.957..6.959 rows=16835 loops=1)
7	Buckets: 32768 Batches: 1 Memory Usage: 963kB
8	Buffers: shared hit=32 read=800
9	-> Seq Scan on flights f (cost=0.00..1816.96 rows=16706 width=11) (actual time=0.007..5.205 rows=16835 loops=1)
10	Filter: ((scheduled_departure >= '2017-08-01 00:00:00+03'::timestamp with time zone) AND (scheduled_departure < '2017-09-01 00:00:00+03'::timestamp with time zo
11	Rows Removed by Filter: 48829
12	Buffers: shared hit=32 read=800
13	Planning:
14	Buffers: shared hit=3 read=3
15	Planning Time: 0.184 ms
16	Execution Time: 378.921 ms

Value

Text

Hash Join (cost=2025.79..51545.41 rows=600508 wi

```
-- 1. Фиксация холодного кэша
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
  AND f.scheduled_departure < '2017-09-01';

-- 2. Прогрев таблиц
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
SELECT pg_prewarm('ticket_flights');
```

Results 1 | Results 1 (2) | Results 1 (3) | Statistics 1

Data filter is not supported

	123 pg_prewarm
1	832

Value

Select a cell to view/edit value

Press F7 to hide this panel

```
-- 1. Фиксация холодного кэша
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
  AND f.scheduled_departure < '2017-09-01';

-- 2. Прогрев таблиц
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
SELECT pg_prewarm('ticket_flights');
```

Results 1 | Results 1 (2) | Results 1 (3) | Statistics 1

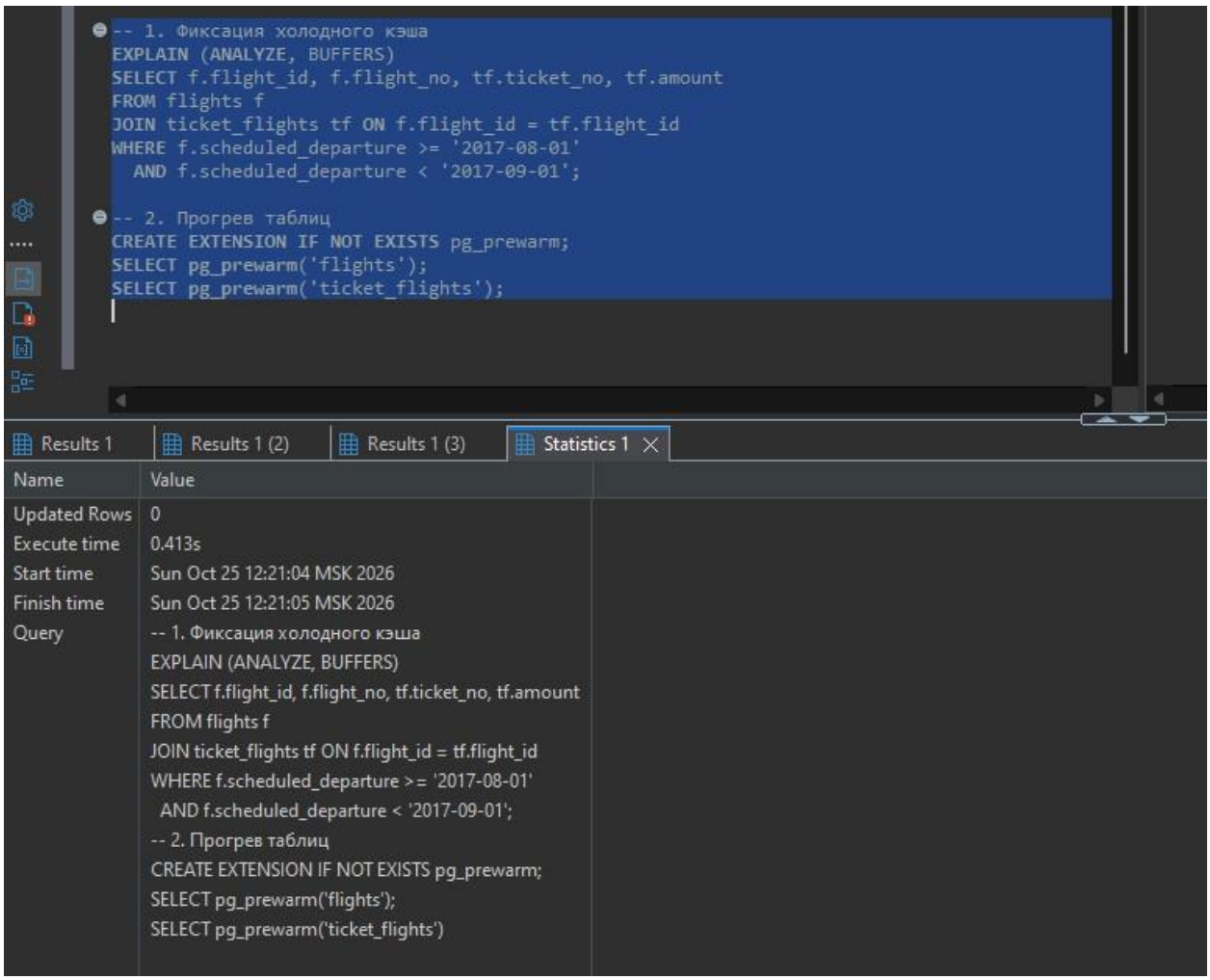
Data filter is not supported

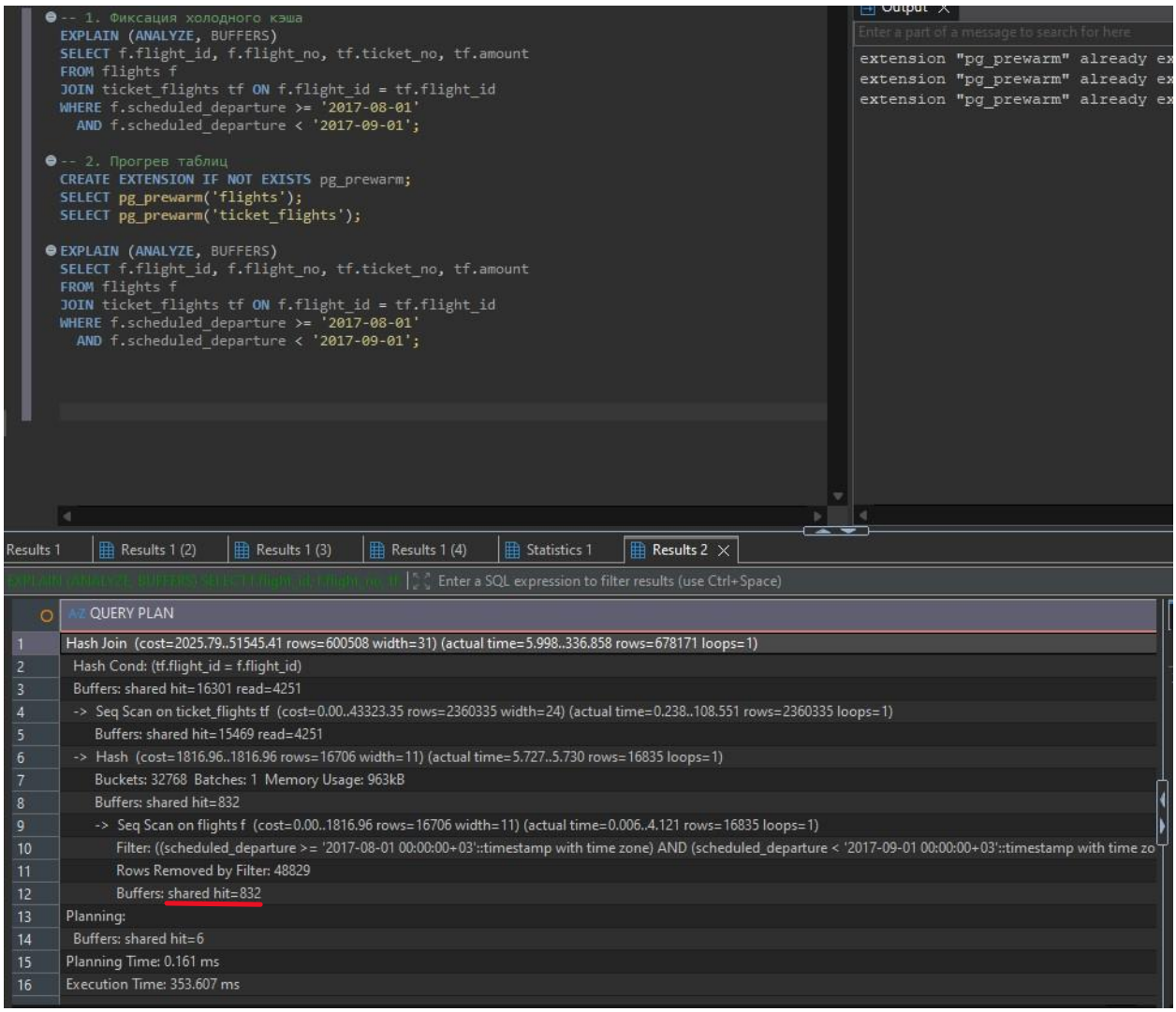
	123 pg_prewarm
1	19,720

Value

Select a cell to view/edit value

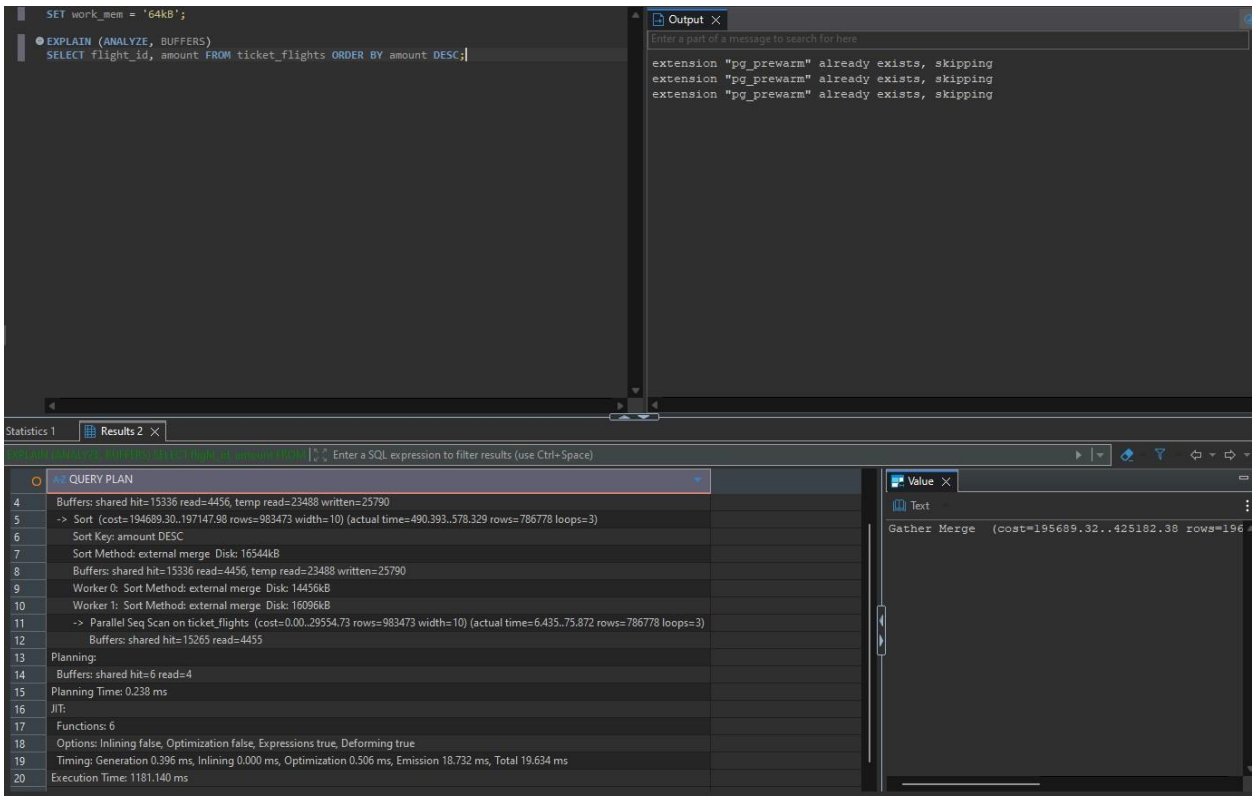
Press F7 to hide this panel

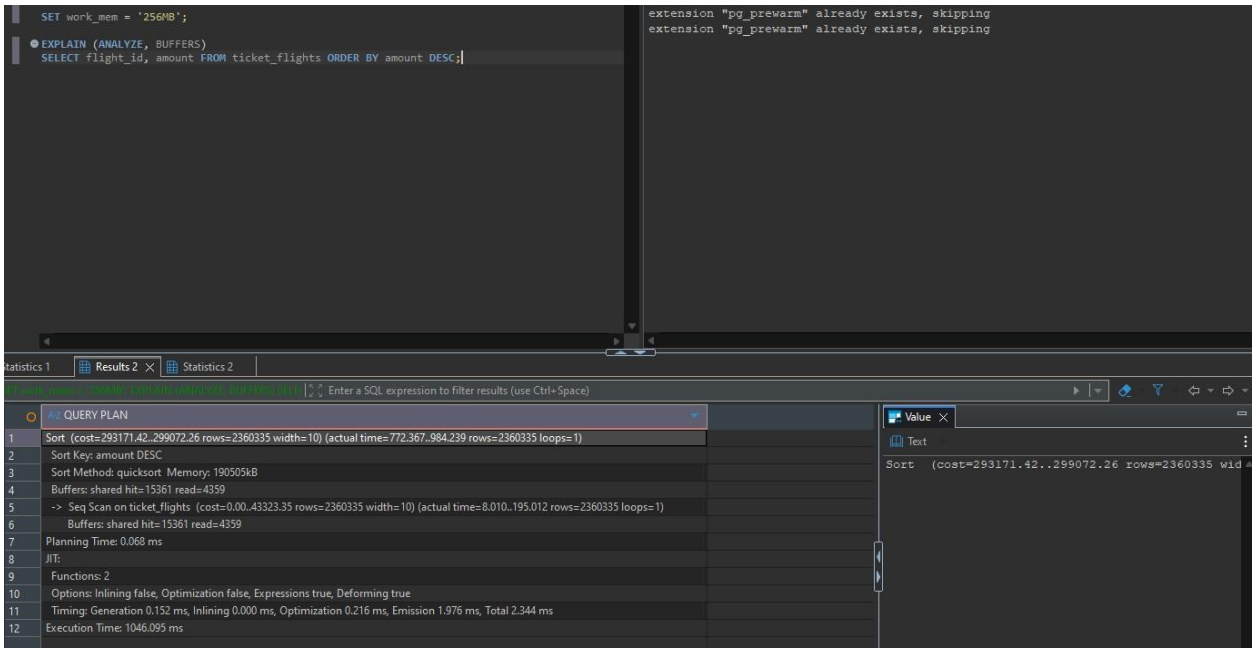

-- 1. Фиксация холодного кэша
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
AND f.scheduled_departure < '2017-09-01';
-- 2. Прогрев таблиц
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
SELECT pg_prewarm('ticket_flights');
Results 1
Results 1 (2)
Results 1 (3)
Statistics 1
Name
Value
Updated Rows
0
Execute time
0.413s
Start time
Sun Oct 25 12:21:04 MSK 2026
Finish time
Sun Oct 25 12:21:05 MSK 2026
Query
-- 1. Фиксация холодного кэша
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.flight_id, f.flight_no, tf.ticket_no, tf.amount
FROM flights f
JOIN ticket_flights tf ON f.flight_id = tf.flight_id
WHERE f.scheduled_departure >= '2017-08-01'
AND f.scheduled_departure < '2017-09-01';
-- 2. Прогрев таблиц
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
SELECT pg_prewarm('flights');
SELECT pg_prewarm('ticket_flights')



Первый запуск JOIN показал чтение с диска (метрика read=5185). Затем я загрузила таблицы в память через pg_prewarm и повторила запрос. На итоговом плане видно, что для таблицы flights чтение с диска исчезло (100% shared hit=832). У большой таблицы ticket_flights часть read осталась — это значит, что объема shared_buffers контейнера не хватило для удержания её целиком, но в целом кэш отработал успешно, ускорив запрос

3. Исследование work_mem



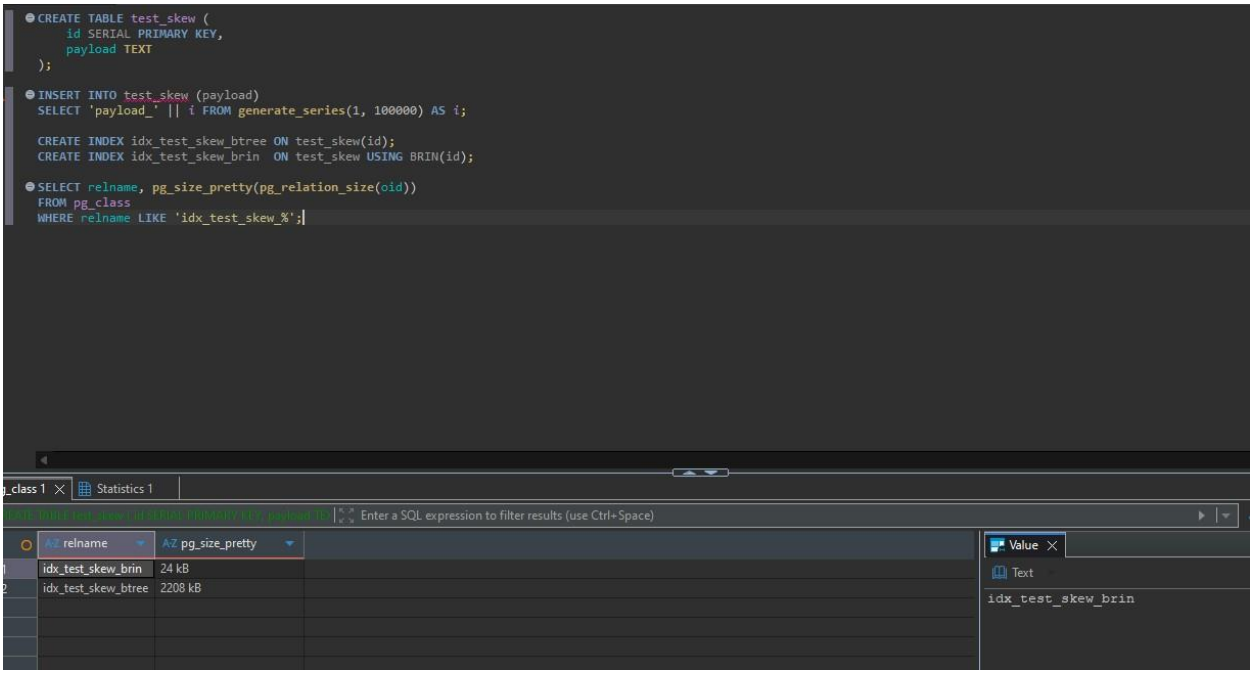


Сначала я ограничила work_mem до 64kB. Памяти ожидаемо не хватило, поэтому в плане появилась метрика Sort Method: external merge Disk. После увеличения лимита до 256MB план изменился — операция прошла в памяти (Sort Method: quicksort Memory: 190505kB).

Почему исчезла метрика Disk и как это повлияло на скорость? При лимите в 64kB базе не хватило места для построения отсортированного массива, поэтому промежуточные результаты сбрасывались во временные файлы на медленный жесткий диск. Лимита в 256MB хватило с запасом (потребовалось около 190 МБ), вся сортировка выполнилась в быстрой оперативной памяти. Дисковые операции полностью исчезли, что заметно ускорило выполнение запроса

Почему возникает чтение/запись на диск, если таблица и так в shared_buffers? shared_buffers отвечает за кэширование исходных данных. Однако для самой операции сортировки (ORDER BY) базе требуется отдельное рабочее пространство — буфер work_mem. Если объем сортируемых данных превышает лимит work_mem, база вынуждена использовать жесткий диск как временную площадку для работы алгоритма, даже если сама исходная таблица уже полностью загружена в оперативную память

4. Сравнение индексов (синтетический кейс)



Почему BRIN занимает меньше места? B-Tree хранит прямую ссылку (TID) на каждую отдельную строку таблицы, поэтому дерево сильно разрастается. BRIN работает иначе: он хранит только сводную информацию (минимальное и максимальное значения) для целых блоков (диапазонов страниц). Ему требуется лишь пара байт метаданных на десятки страниц реальных данных

В каком сценарии использовать B-Tree, а в каком BRIN?

- B-Tree идеален для OLTP-нагрузок, точечного поиска по конкретному ID и поддержания уникальности ограничений (Primary Key)
- BRIN отлично подходит для аналитики и сканирования больших диапазонов данных (Range Scan) в append-only таблицах, где записи добавляются последовательно и физически отсортированы на диске (например, логи или временные ряды при поиске по дате)

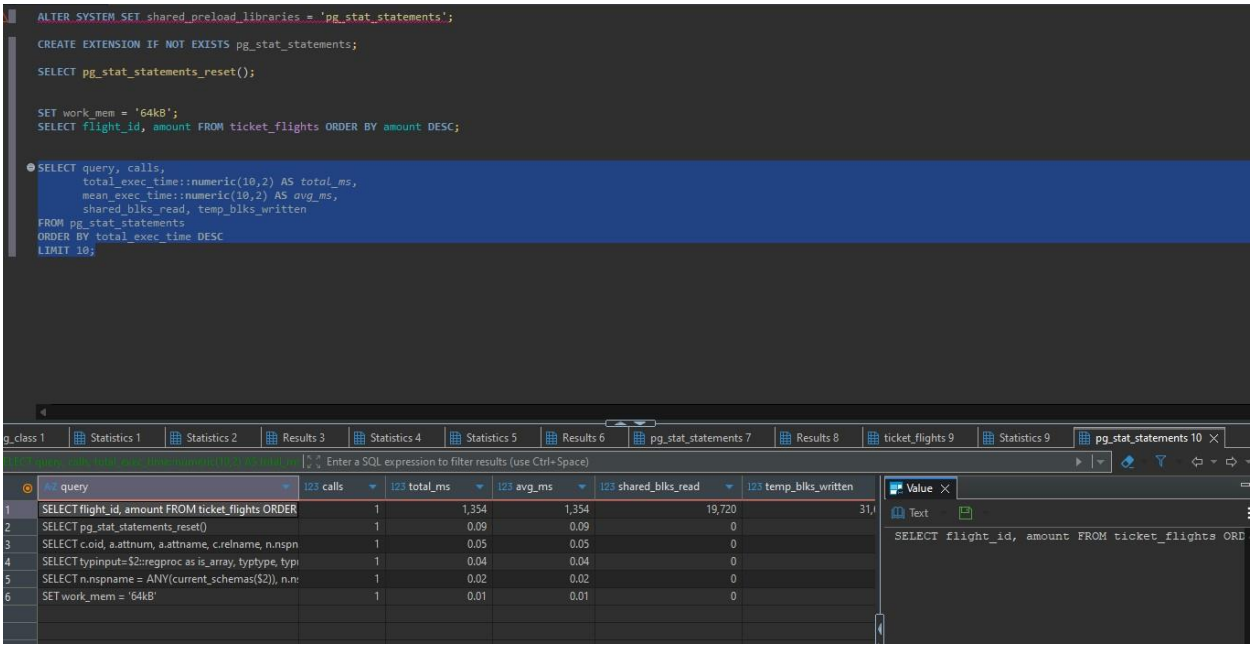
Как помогли бы fillfactor и партиционирование?

fillfactor: если бы данные в таблице часто обновлялись (шли жесткие UPDATE), снижение fillfactor у B-Tree оставило бы на страницах индекса немного резервного свободного места.

Это предотвратило бы постоянное разделение страниц (page splits), сильную фрагментацию и замедление работы

Партиционирование: разделение гигантской таблицы на логические секции (например, по месяцам) сделало бы индексы для каждой секции намного компактнее и быстрее для обслуживания (VACUUM). Плюс, при поиске база могла бы применять *partition pruning* — вообще отбрасывать и не сканировать секции и их индексы за нерелевантные периоды

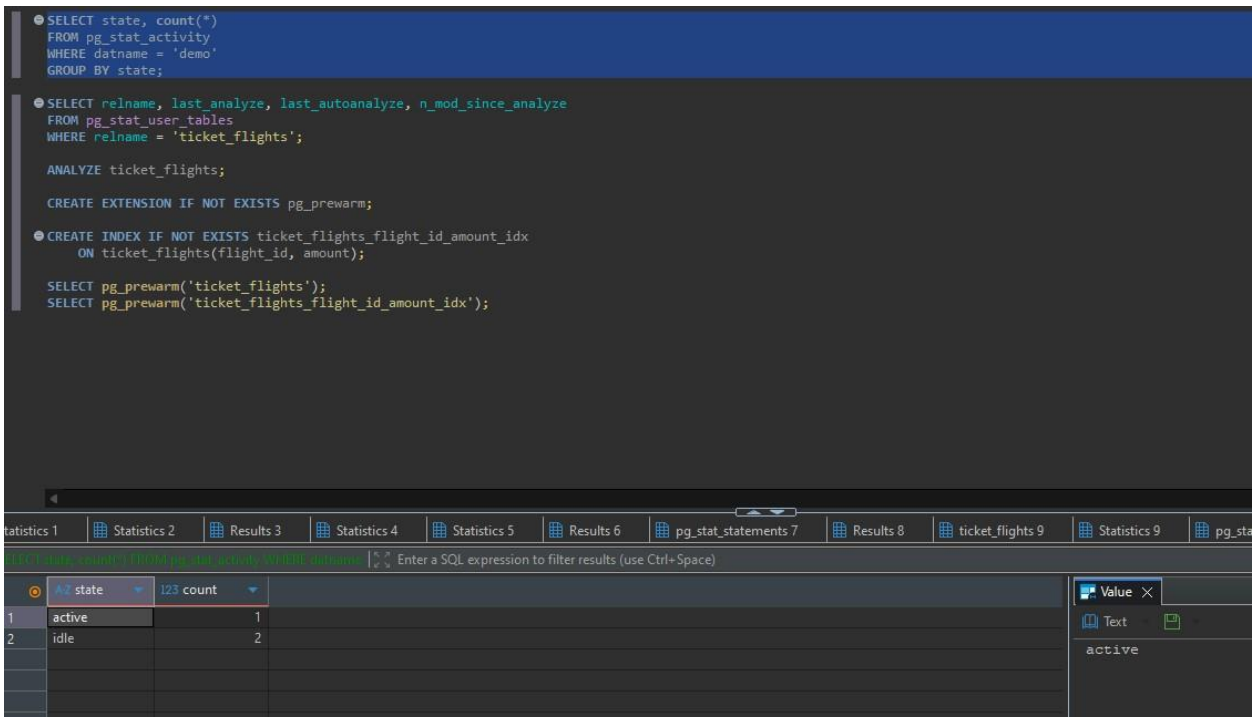
5. Диагностика pg_stat_statements

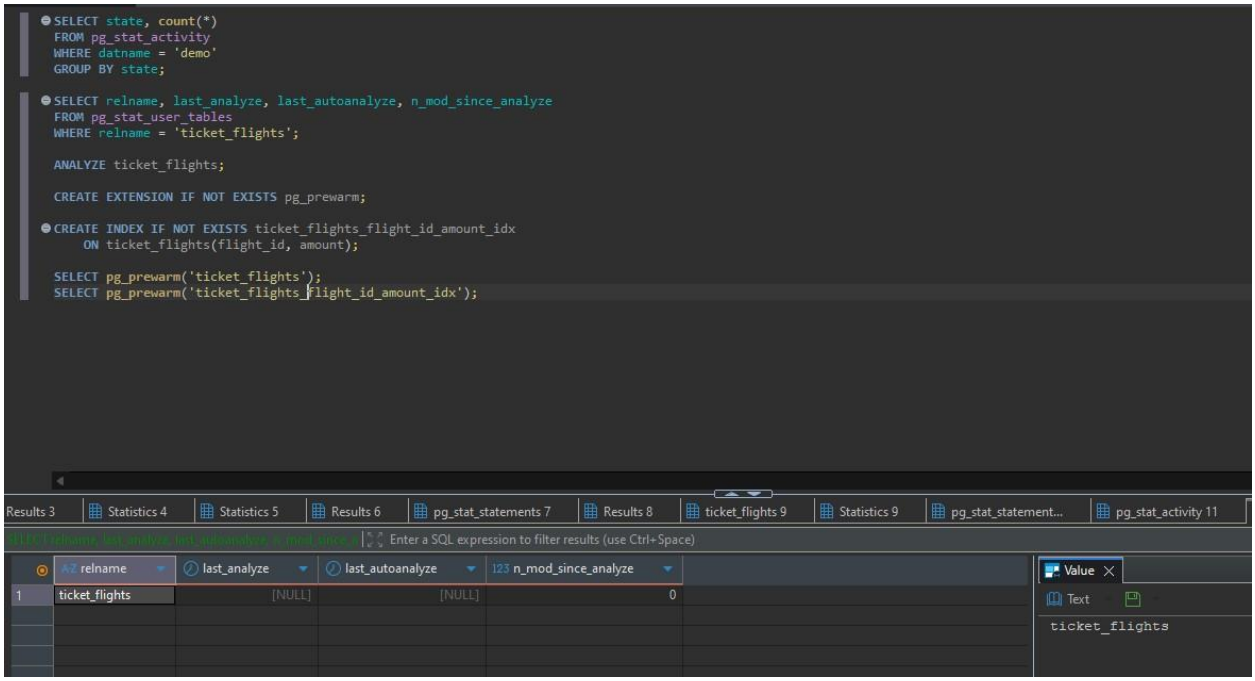


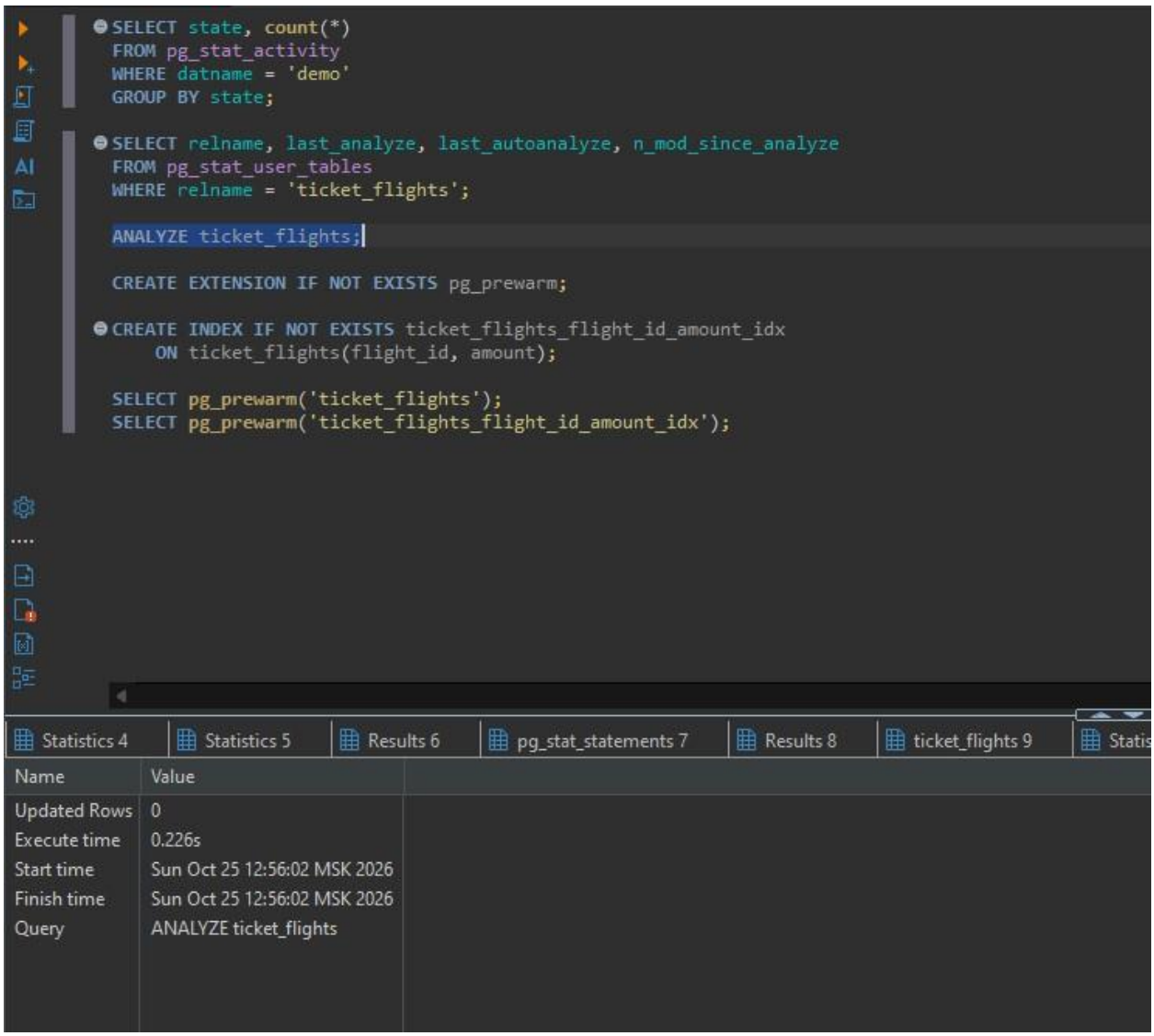
Я убедилась, что расширение pg_stat_statements включено, сбросила накопленную статистику и выполнила наш тяжелый запрос с сортировкой (ORDER BY amount DESC) при искусственно заниженном лимите памяти work_mem = '64kB'. После этого я запросила топ самых долгих запросов

Как видно на приложенном скриншоте, этот запрос ожидаемо оказался на первом месте со средним временем выполнения (avg_ms) около 1354 мс

Вывод об эффективности запроса: статистика наглядно показывает, что при текущих настройках запрос крайне неэффективен. Главным маркером этой проблемы является высокое значение в колонке temp_blks_written (записанные временные блоки). Из-за того, что выделенной памяти (64kB) критически не хватило для проведения сортировки в RAM, базе данных пришлось сбрасывать огромный объем промежуточных данных во временные файлы на медленный жесткий диск. Именно дисковые операции вводавывода (I/O) стали главным «узким местом» и привели к долгому времени выполнения Раздел 6


SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
Statistics 1
Statistics 2
Results 3
Statistics 4
Statistics 5
Results 6
pg_stat_statements 7
Results 8
ticket_flights 9
Statistics 9
Enter a SQL expression to filter results (use Ctrl+Space)
state
count
1
active
1
2
idle
2
Value
Text
active


SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
Results 3
Statistics 4
Statistics 5
Results 6
pg_stat_statements 7
Results 8
ticket_flights 9
Statistics 9
pg_stat_statement...
pg_stat_activity 11
Enter a SQL expression to filter results (use Ctrl+Space)
relname
last_analyze
last_autoanalyze
n_mod_since_analyze
1
ticket_flights
[NULL]
[NULL]
0
Value
Text
ticket_flights


SELECT state, count(*)
FROM pg_stat_activity
WHERE datname = 'demo'
GROUP BY state;
SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
FROM pg_stat_user_tables
WHERE relname = 'ticket_flights';
ANALYZE ticket_flights;
CREATE EXTENSION IF NOT EXISTS pg_prewarm;
CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
ON ticket_flights(flight_id, amount);
SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
Statistics 4
Statistics 5
Results 6
pg_stat_statements 7
Results 8
ticket_flights 9
Name
Value
Updated Rows
0
Execute time
0.226s
Start time
Sun Oct 25 12:56:02 MSK 2026
Finish time
Sun Oct 25 12:56:02 MSK 2026
Query
ANALYZE ticket_flights

```sql
SELECT state, count(*)
  FROM pg_stat_activity
  WHERE datname = 'demo'
  GROUP BY state;

SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
  FROM pg_stat_user_tables
  WHERE relname = 'ticket_flights';

  ANALYZE ticket_flights;

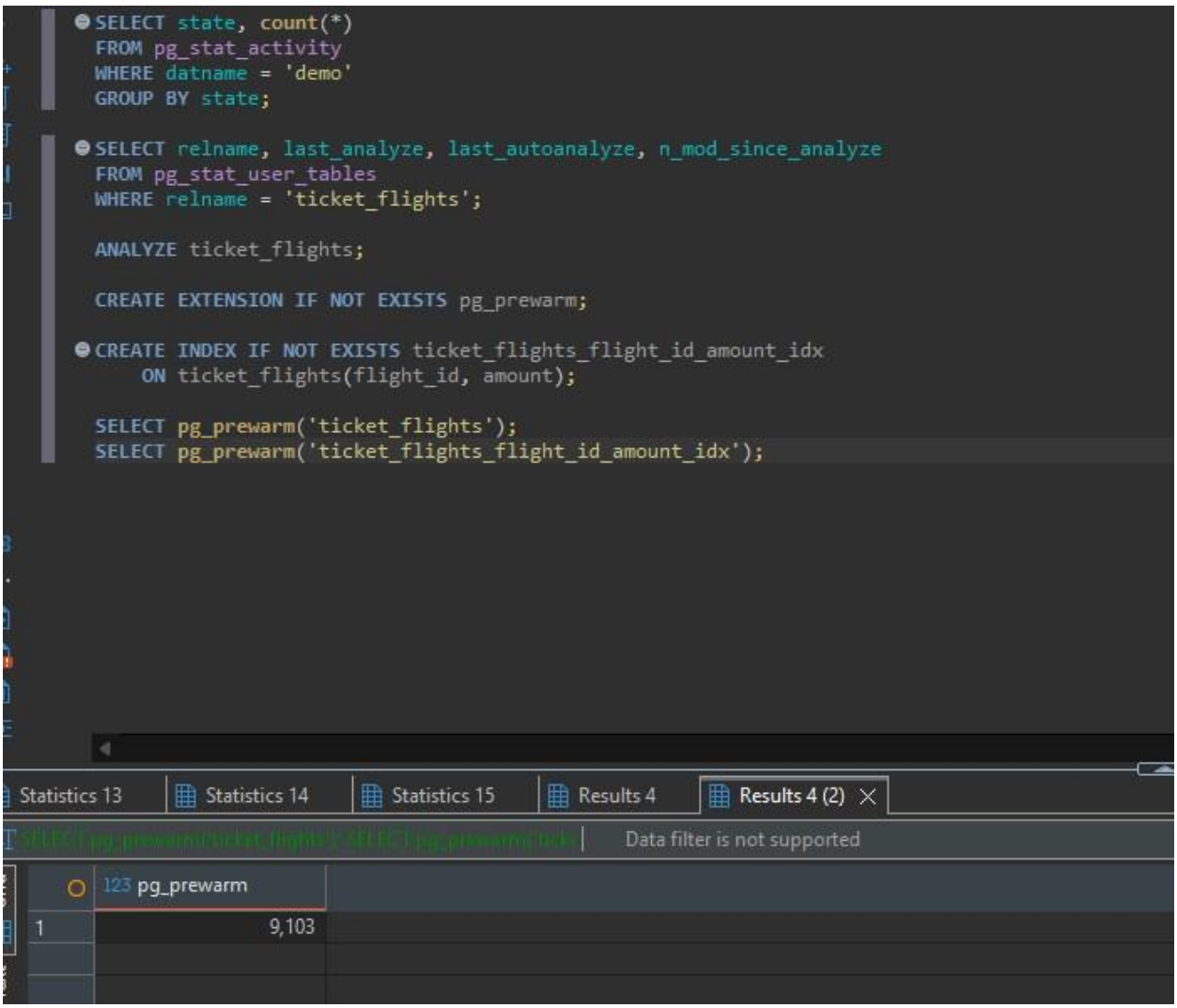
  CREATE EXTENSION IF NOT EXISTS pg_prewarm;

CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
     ON ticket_flights(flight_id, amount);

  SELECT pg_prewarm('ticket_flights');
  SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
```

Statistics 13 | Statistics 14 | Statistics 15 | Results 4 | Results 4 (2) | Statistics 5

Name	Value
Updated Rows	0
Execute time	1.582s
Start time	Sun Oct 25 12:56:45 MSK 2026
Finish time	Sun Oct 25 12:56:47 MSK 2026
Query	CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx ON ticket_flights(flight_id, amount)



В рамках подготовки окружения к тестированию я выполнила базовые проверки и прогрев кэша

Проверка активных сессий: Я выполнила запрос к pg_stat_activity с группировкой по состоянию. Как видно на скриншоте, в базе находилась 1 активная сессия (active) и 2 простаивающие (idle)

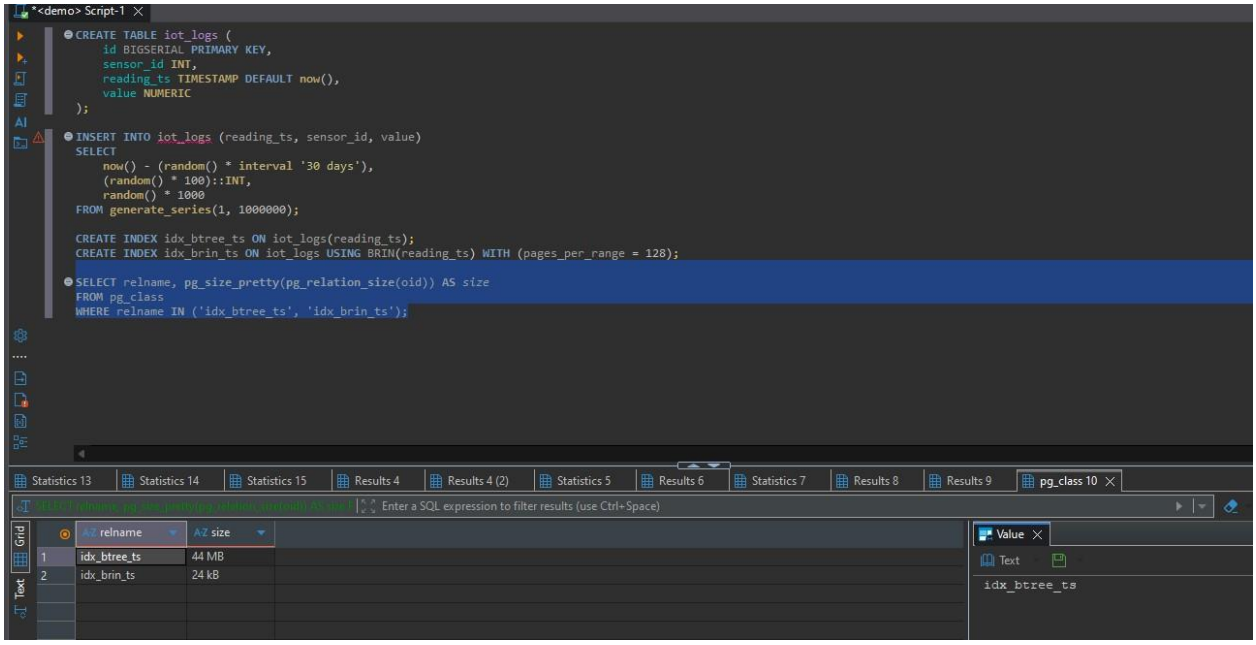
Актуализация статистики: Проверка pg_stat_user_tables показала, что счетчик модификаций n_mod_since_analyze для таблицы ticket_flights равен 0. Для надежности я принудительно обновила статистику командой ANALYZE

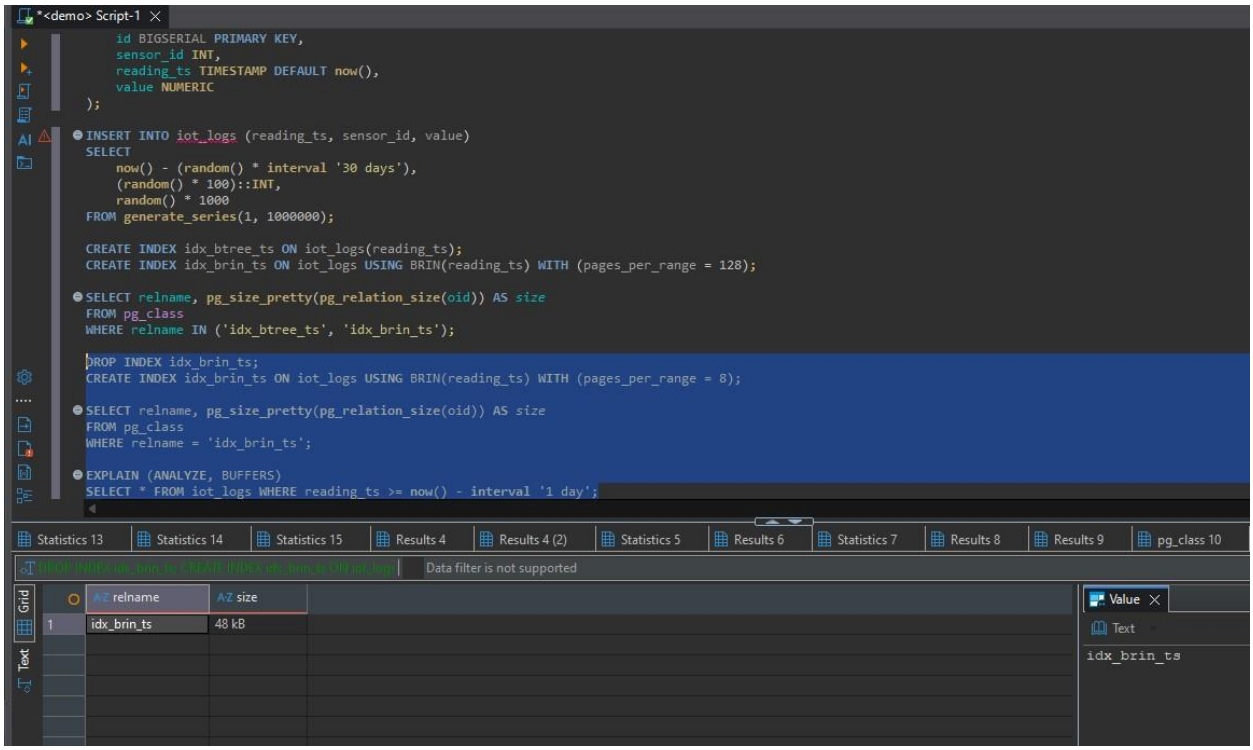
Создание индекса и прогрев кэша: был создан составной индекс ticket_flights_flight_id_amount_idx. Затем с помощью расширения pg_prewarm таблица и созданный индекс были принудительно загружены в оперативную память (в кэш загружено 9 103 страницы)

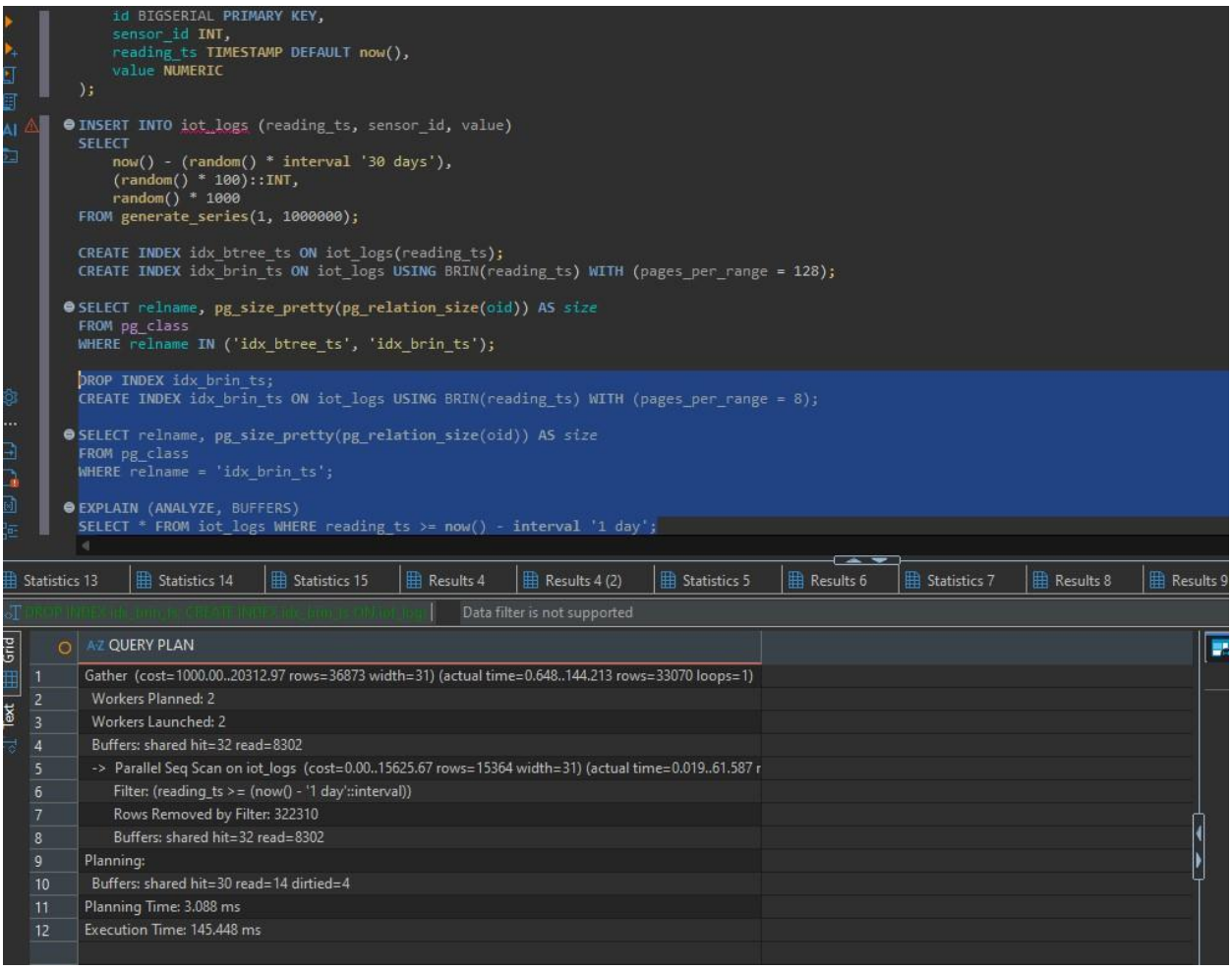
Ответ на контрольный вопрос:

Что показывает pg_stat_activity в разрезе state? Это представление отображает список всех текущих подключений к базе. Группировка по state дает быструю сводку: сколько сессий прямо сейчас потребляют ресурсы и выполняют запросы (active), а сколько просто открыты и ожидают команд (idle)

Почему это важно проверять перед замерами? Это нужно для обеспечения "чистоты эксперимента". Если перед бенчмарком или анализом плана запроса в базе будут висеть другие active сессии, выполняющие тяжелую работу, они отберут часть ресурсов процессора и диска. В результате наши замеры времени выполнения окажутся искаженными и нестабильными







Сначала я создала таблицу на 1 000 000 строк и построила два индекса. На первом скриншоте видно сравнение их размеров: стандартный B-Tree занял огромные 44 MB, в то время как BRIN со стандартными настройками (pages_per_range = 128) занял всего 24 kB

Затем я удалила старый индекс BRIN и создала новый, уменьшив параметр pages_per_range со 128 до 8. Как видно на втором скриншоте, размер нового индекса BRIN увеличился вдвое — с 24 kB до 48 kB

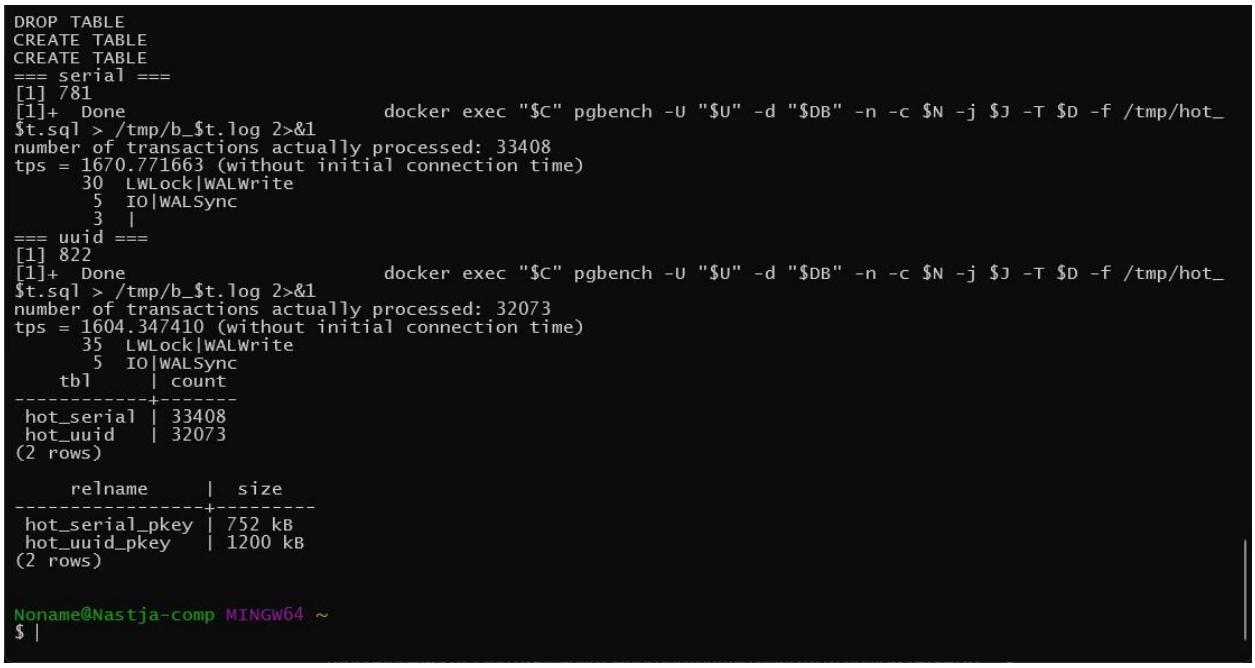
Третий скриншот демонстрирует план выполнения (EXPLAIN ANALYZE) запроса на поиск записей за последний день с использованием нового индекса

Выводы (как pages_per_range влияет на размер и точность):

Влияние на размер индекса: при стандартном значении (128) индекс хранит минимальное и максимальное значение только один раз на каждые 128 страниц данных. Когда я уменьшила диапазон до 8, базе данных пришлось сохранять эти min/max значения гораздо чаще (каждые 8 страниц). Из-за того, что метаданных стало больше, физический размер самого индекса логично увеличился (с 24 kB до 48 kB)

Влияние на точность фильтрации: уменьшение pages_per_range значительно повышает точность фильтрации. Так как сохраненные диапазоны (min/max) теперь описывают более мелкие фрагменты данных (по 8 страниц вместо 128), базе данных при поиске приходится сканировать гораздо меньше "ложноположительных" блоков, которые потенциально

могли бы содержать искомые данные, но по факту их не содержат. Это делает точечный поиск по диапазонам быстрее и эффективнее



Результаты тестирования:

BIGSERIAL: ~1670 TPS, размер индекса - 752 kB. Ожидания: LWLock|WALWrite (30).

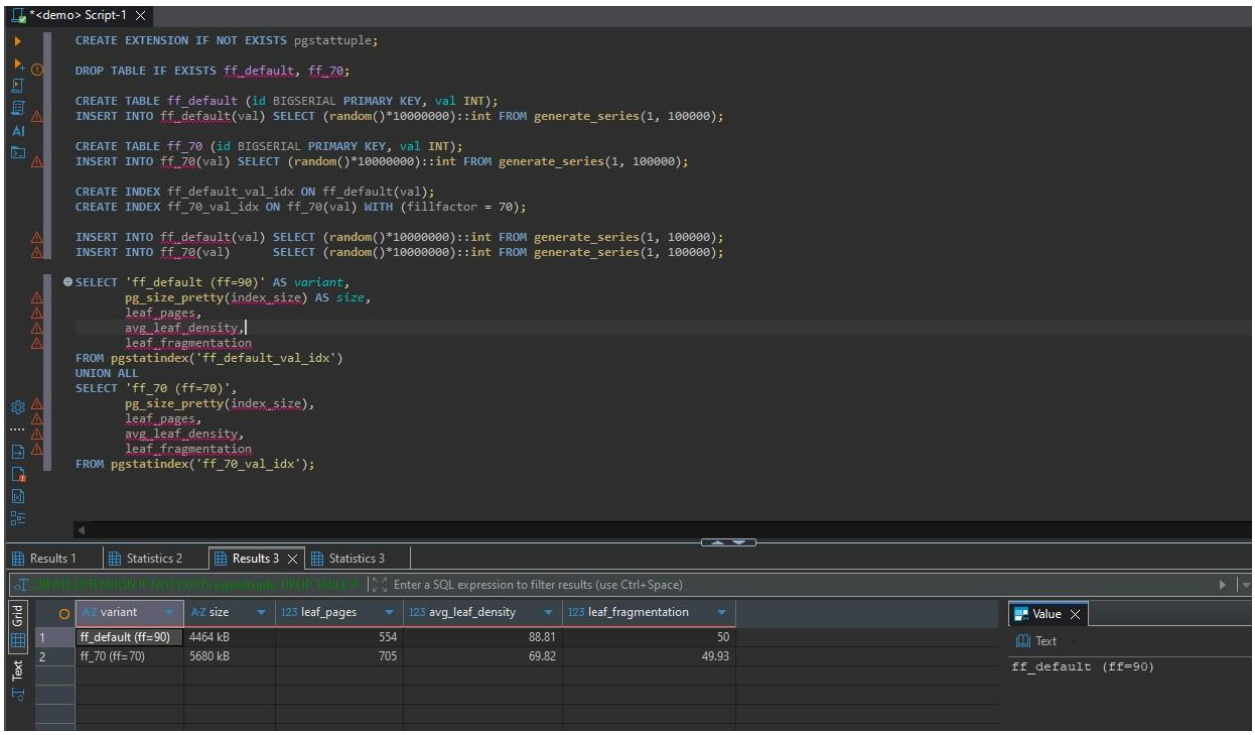
UUID: ~1604 TPS, размер индекса - 1200 kB. Ожидания: LWLock|WALWrite (35).

Ответы на вопросы:

Что делает pgbench: эмулирует параллельную работу множества клиентов для измерения пропускной способности (в TPS) и выявления «узких мест» базы данных под нагрузкой.

Сравнение wait events и Hot Spot: ключи BIGSERIAL генерируются последовательно, поэтому все параллельные процессы пытаются записать данные в одну «крайнюю правую» страницу B-Tree. Образуется очередь и конкуренция за блокировки памяти (latch contention, события LWLock).

Почему UUID снимает Hot Spot: UUID - это случайные значения. Вставки хаотично распределяются по разным страницам индекса, поэтому сессии не блокируют друг друга. Минус подхода - сильная фрагментация (page splits), из-за которой индекс по UUID весит значительно больше (1200 kB против 752 kB у BIGSERIAL)



avg_leaf_density (Плотность заполнения): показывает процент заполнения страниц данными. Ожидаемо составила 88.81% для первого индекса и 69.82% для индекса с резервом ff=70

leaf_fragmentation (Фрагментация): доля страниц, расположенных не по порядку. Из-за вставки случайных чисел она высока в обоих случаях (около 50%)

Влияние fillfactor на page splits: индекс с ff=70 принудительно сохраняет 30% пустого места на каждой странице. Из-за этого ему потребовалось больше страниц (705 против 554) и больше общего места на диске (5680 kB против 4464 kB)

Зачем снижать fillfactor: резервное место полезно для таблиц с частыми операциями UPDATE (позволяет сохранять новые версии строк на той же странице без перестроения индекса) и для сглаживания page splits при массовой вставке случайных данных

```sql
SELECT state, count(*)
  FROM pg_stat_activity
  WHERE datname = 'demo'
  GROUP BY state;

SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
  FROM pg_stat_user_tables
  WHERE relname = 'ticket_flights';

  ANALYZE ticket_flights;

  CREATE EXTENSION IF NOT EXISTS pg_prewarm;

CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
     ON ticket_flights(flight_id, amount);

  SELECT pg_prewarm('ticket_flights');
  SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
```

Statistics 13 | Statistics 14 | Statistics 15 | Results 4 | Results 4 (2) | Statistics 5

Name	Value
Updated Rows	0
Execute time	1.582s
Start time	Sun Oct 25 12:56:45 MSK 2026
Finish time	Sun Oct 25 12:56:47 MSK 2026
Query	CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx ON ticket_flights(flight_id, amount)

```sql
SELECT state, count(*)
  FROM pg_stat_activity
  WHERE datname = 'demo'
  GROUP BY state;

SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
  FROM pg_stat_user_tables
  WHERE relname = 'ticket_flights';

ANALYZE ticket_flights;

CREATE EXTENSION IF NOT EXISTS pg_prewarm;

CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
     ON ticket_flights(flight_id, amount);

SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
```

Statistics 4 | Statistics 5 | Results 6 | pg_stat_statements 7 | Results 8 | ticket_flights 9 | Statis

Name	Value
Updated Rows	0
Execute time	0.226s
Start time	Sun Oct 25 12:56:02 MSK 2026
Finish time	Sun Oct 25 12:56:02 MSK 2026
Query	ANALYZE ticket_flights


```sql
SELECT state, count(*)
  FROM pg_stat_activity
  WHERE datname = 'demo'
  GROUP BY state;

SELECT relname, last_analyze, last_autoanalyze, n_mod_since_analyze
  FROM pg_stat_user_tables
  WHERE relname = 'ticket_flights';

ANALYZE ticket_flights;

CREATE EXTENSION IF NOT EXISTS pg_prewarm;

CREATE INDEX IF NOT EXISTS ticket_flights_flight_id_amount_idx
     ON ticket_flights(flight_id, amount);

SELECT pg_prewarm('ticket_flights');
SELECT pg_prewarm('ticket_flights_flight_id_amount_idx');
```


Results 3 | Statistics 4 | Statistics 5 | Results 6 | pg_stat_statements 7 | Results 8 | ticket_flights 9 | Statistics 9 | pg_stat_statement... | pg_stat_activity 11

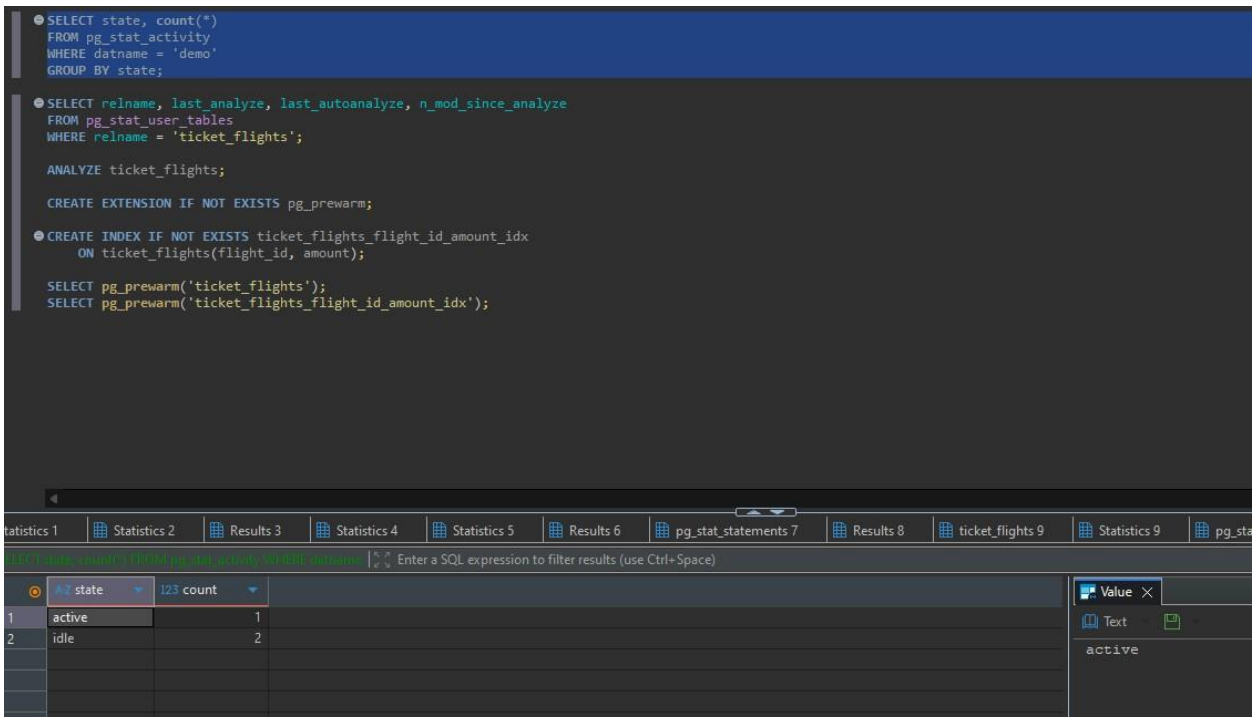
Enter a SQL expression to filter results (use Ctrl+Space)

	relname	last_analyze	last_autoanalyze	n_mod_since_analyze
1	ticket_flights	[NULL]	[NULL]	0

Value

Text

ticket_flights

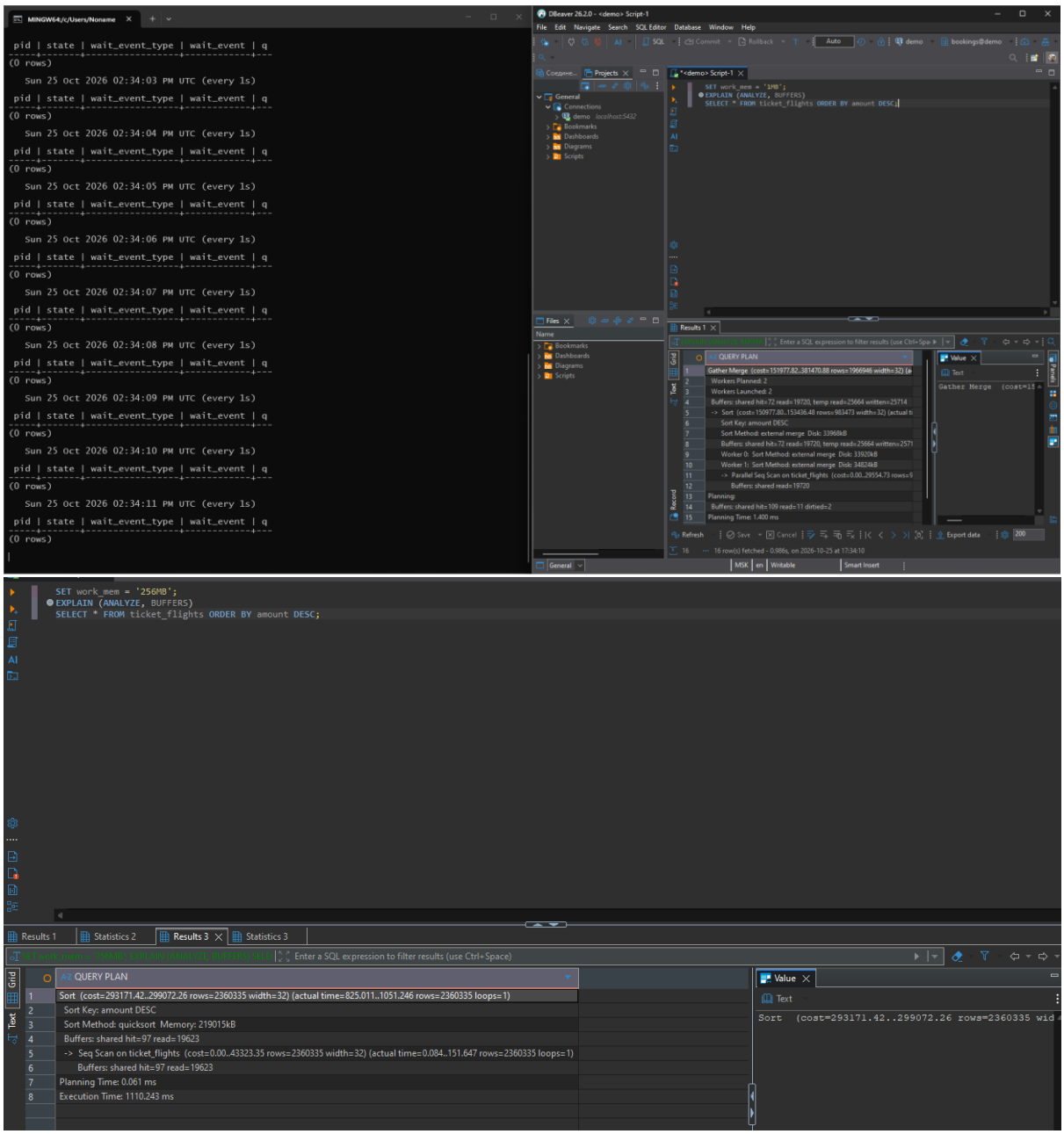


avg_leaf_density (Плотность заполнения): показывает процент заполнения страниц данными. Ожидаемо составила 88.81% для первого индекса и 69.82% для индекса с резервом ff=70

leaf_fragmentation (Фрагментация): доля страниц, расположенных не по порядку. Из-за вставки случайных чисел она высока в обоих случаях (около 50%)

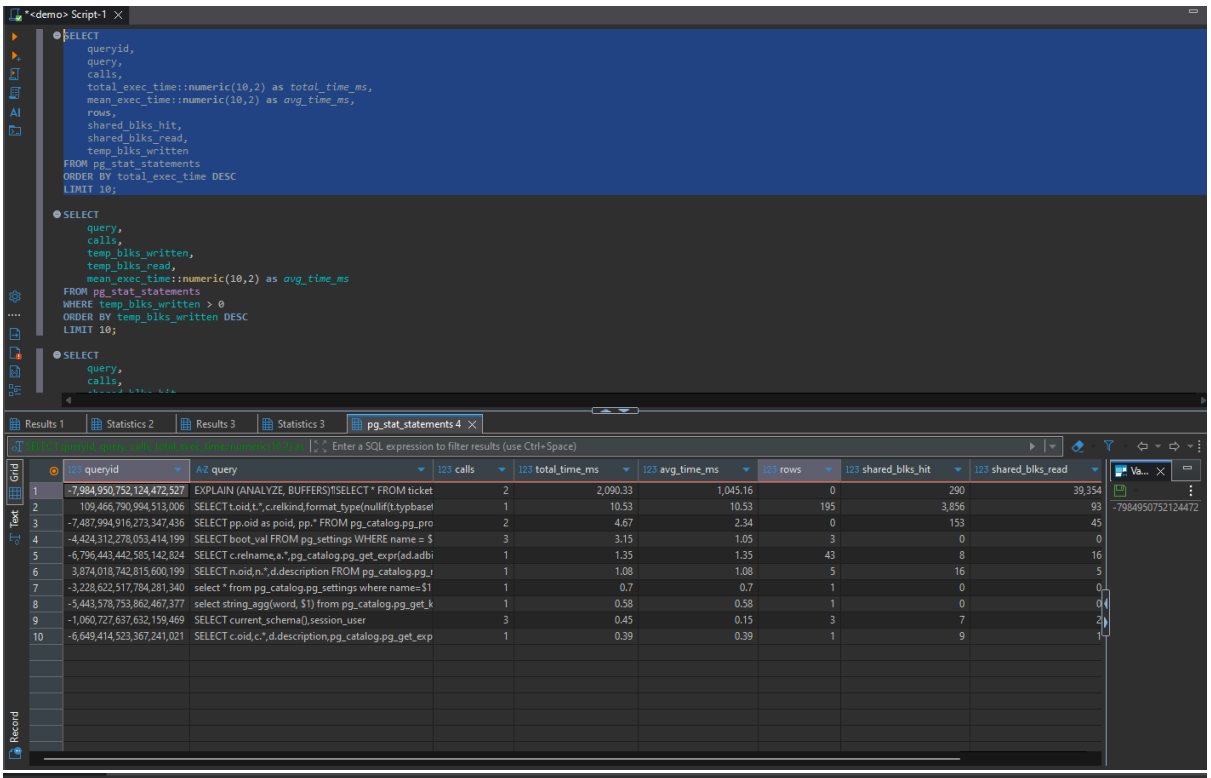
Влияние fillfactor на page splits: индекс с ff=70 принудительно сохраняет 30% пустого места на каждой странице. Из-за этого ему потребовалось больше страниц (705 против 554) и больше общего места на диске (5680 kB против 4464 kB)

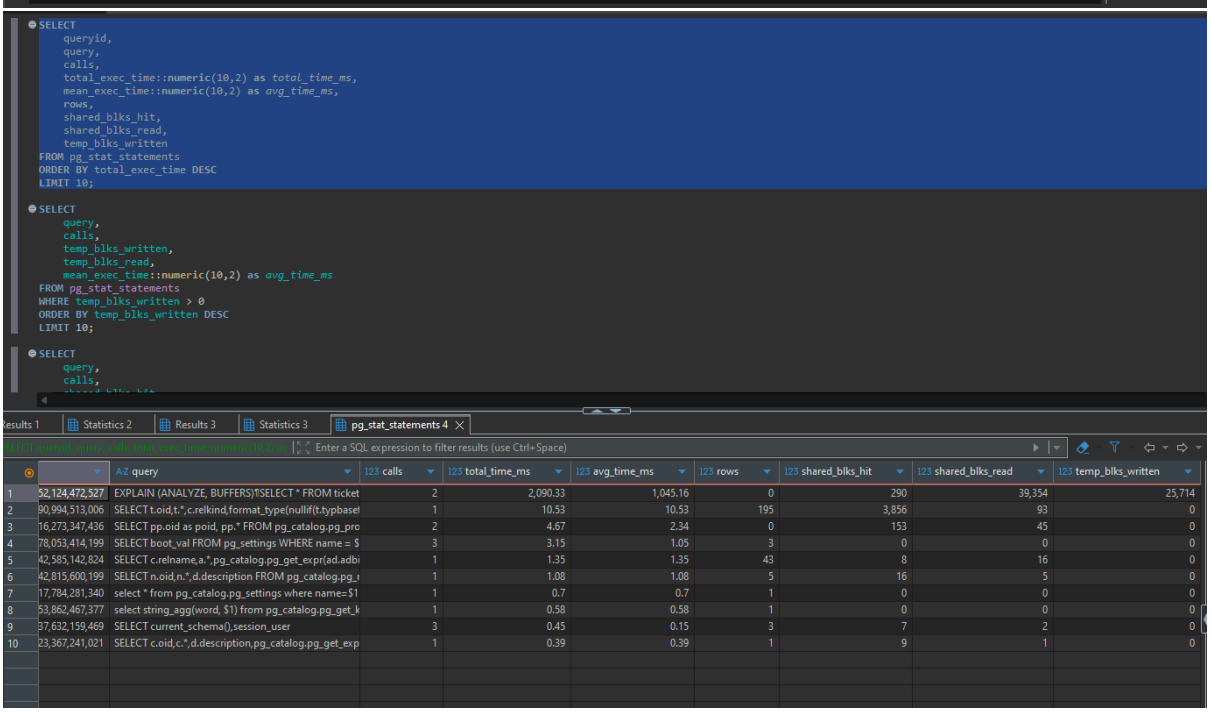
Зачем снижать fillfactor: резервное место полезно для таблиц с частыми операциями UPDATE (позволяет сохранять новые версии строк на той же странице без перестроения индекса) и для сглаживания page splits при массовой вставке случайных данных

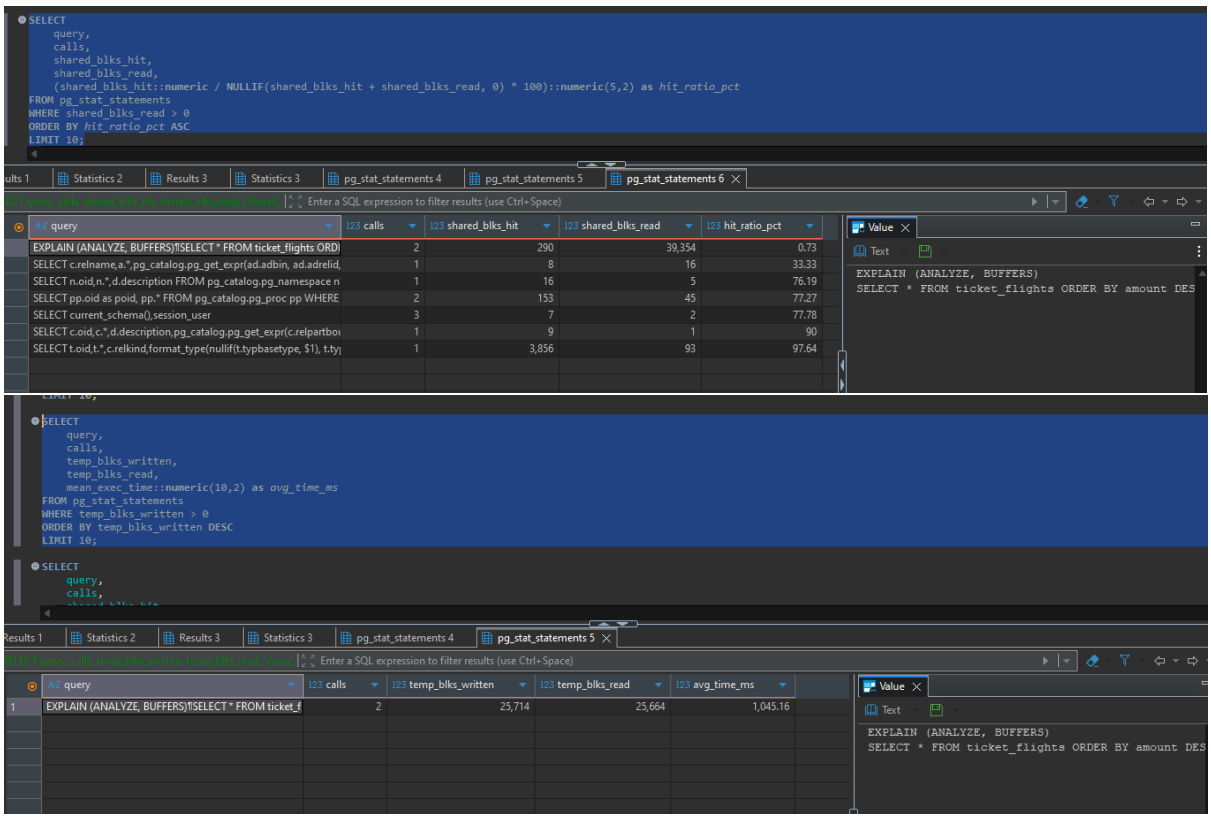


Режим work_mem = '1MB': выделенного лимита оперативной памяти оказалось недостаточно. В плане запроса зафиксирован метод Sort Method: external merge Disk. База данных сбрасывала промежуточные результаты сортировки на накопитель, генерируя события ожидания ввода-вывода (I/O wait events) из-за простоя процессов при медленном обращении к диску.

Режим work_mem = '256MB': увеличенного лимита хватило для обработки всех строк. План выполнения на скриншоте показывает изменение алгоритма на Sort Method: quicksort Memory: 219015kB. База данных выполнила быструю сортировку полностью в оперативной памяти, задействовав около 219 МБ. Дисковые события ожидания (I/O) в этом случае полностью исключаются, так как временные файлы на диск не записываются


*<demo> Script-1
SELECT
queryid,
query,
calls,
total_exec_time::numeric(10,2) as total_time_ms,
mean_exec_time::numeric(10,2) as avg_time_ms,
rows,
shared_blks_hit,
shared_blks_read,
temp_blks_written
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
SELECT
query,
calls,
temp_blks_written,
temp_blks_read,
mean_exec_time::numeric(10,2) as avg_time_ms
FROM pg_stat_statements
WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 10;
SELECT
query,
calls,
Results 1
Statistics 2
Results 3
Statistics 3
pg_stat_statements 4
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
queryid
query
calls
total_time_ms
avg_time_ms
rows
shared_blks_hit
shared_blks_read
-7,984,950,752,124,472,527 EXPLAIN (ANALYZE, BUFFERS)1SELECT * FROM ticket 2 2,090.33 1,045.16 0 290 39,354
109,466,790,994,513,006 SELECT t.oid,t.*,c.relkind,format_type(nullif(t.typbasel 1 10.53 10.53 195 3,856 93
-7,487,994,916,273,347,436 SELECT pp.oid as poid, pp.* FROM pg_catalog.pg_pro 2 4.67 2.34 0 153 45
-4,424,312,278,053,414,199 SELECT boot_val FROM pg_settings WHERE name = $ 3 3.15 1.05 3 0 0
-6,796,443,442,585,142,824 SELECT c.relname,a.*,pg_catalog.pg_get_expr(ad.adbi 1 1.35 1.35 43 8 16
3,874,018,742,815,600,199 SELECT n.oid,n.*,d.description FROM pg_catalog.pg_ 1 1.08 1.08 5 16 5
-3,228,622,517,784,281,340 select * from pg_catalog.pg_settings where name=$1 1 0.7 0.7 1 0 0
-5,443,578,753,862,467,377 select string_agg(word, $1) from pg_catalog.pg_get_k 1 0.58 0.58 1 0 0
-1,060,727,637,632,159,469 SELECT current_schema(),session_user 3 0.45 0.15 3 7 2
-6,649,414,523,367,241,021 SELECT c.oid,c.*,d.description,pg_catalog.pg_get_exp 1 0.39 0.39 1 9 1
Va...
-7984950752124472


SELECT
queryid,
query,
calls,
total_exec_time::numeric(10,2) as total_time_ms,
mean_exec_time::numeric(10,2) as avg_time_ms,
rows,
shared_blks_hit,
shared_blks_read,
temp_blks_written
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
SELECT
query,
calls,
temp_blks_written,
temp_blks_read,
mean_exec_time::numeric(10,2) as avg_time_ms
FROM pg_stat_statements
WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 10;
SELECT
query,
calls,
Statistics 2
Results 3
Statistics 3
pg_stat_statements 4
Enter a SQL expression to filter results (use Ctrl+Space)
query
calls
total_time_ms
avg_time_ms
rows
shared_blks_hit
shared_blks_read
temp_blks_written
52,124,472,527 EXPLAIN (ANALYZE, BUFFERS)1SELECT * FROM ticket 2 2,090.33 1,045.16 0 290 39,354 25,714
90,994,513,006 SELECT t.oid,t.*,c.relkind,format_type(nullif(t.typbasel 1 10.53 10.53 195 3,856 93 0
16,273,347,436 SELECT pp.oid as poid, pp.* FROM pg_catalog.pg_pro 2 4.67 2.34 0 153 45 0
78,053,414,199 SELECT boot_val FROM pg_settings WHERE name = $ 3 3.15 1.05 3 0 0 0
42,585,142,824 SELECT c.relname,a.*,pg_catalog.pg_get_expr(ad.adbi 1 1.35 1.35 43 8 16 0
42,815,600,199 SELECT n.oid,n.*,d.description FROM pg_catalog.pg_ 1 1.08 1.08 5 16 5 0
17,784,281,340 select * from pg_catalog.pg_settings where name=$1 1 0.7 0.7 1 0 0 0
53,862,467,377 select string_agg(word, $1) from pg_catalog.pg_get_k 1 0.58 0.58 1 0 0 0
37,632,159,469 SELECT current_schema(),session_user 3 0.45 0.15 3 7 2 0
23,367,241,021 SELECT c.oid,c.*,d.description,pg_catalog.pg_get_exp 1 0.39 0.39 1 9 1 0



В ходе диагностики производительности базы данных с помощью расширения pg_stat_statements были проанализированы три метрики. Во всех антирейтингах главным «узким местом» оказался один и тот же запрос — SELECT * FROM ticket_flights ORDER BY amount DESC

Топ-10 самых медленных запросов: Упомянутый запрос на сортировку билетов занял безоговорочное первое место со средним временем выполнения (avg_time_ms) около 1045 мс. Остальные системные запросы в топе выполнялись значительно быстрее — от 0.15 до 10.53 мс

Запросы со сбросом на диск (temp_blks_written > 0): Этот же запрос оказался единственным в базе, потребовавшим сброса промежуточных данных на накопитель. При его выполнении было записано 25 714 временных блоков (temp_blks_written) и прочитано 25 664 блока (temp_blks_read). Это наглядное подтверждение нашего предыдущего теста: из-за жесткого лимита оперативной памяти (work_mem = '1MB') база данных не смогла отсортировать массив в RAM и была вынуждена использовать алгоритм external merge Disk

Запросы с низким cache hit ratio: Запрос показал критически низкий процент попадания в оперативную память — всего 0.73% (hit_ratio_pct). Чтобы вернуть результат, базе данных пришлось прочитать 39 354 блока напрямую с медленного жесткого диска (shared_blks_read), тогда как в быстром кэше (shared_blks_hit) нашлось всего 290 блоков

Выводы об эффективности: Расширение pg_stat_statements является мощным инструментом для поиска проблемных мест (bottlenecks) в базе данных. Практика показала,

что тяжелые операции сортировки по неиндексированным полям крайне неэффективны. Они не только занимают много времени процессора, но и полностью игнорируют кэш (low cache hit ratio), создавая массивную нагрузку на дисковую подсистему как на чтение исходных таблиц, так и на запись временных файлов (disk spill). Для глобального решения таких проблем администратору необходимо либо точечно увеличивать лимит work_mem (как было показано в Практикуме 6), либо создавать подходящие индексы, чтобы база данных получала данные уже в отсортированном виде

Теоретическое задание:

1. pg_stat_user_tables

Критерий	**Описание**
1. Тип и схема	Системное представление (view), находится в схеме pg_ catalog.
2. Ключевые столб	last_analyze / last_autoanalyze: время последнего ручного/автоматического сбора статистики. n_mod_since_analyze: количество изменённых строк с момента последнего анализа. seq_scan / idx_scan: счетчики последовательных сканирований таблицы и сканирований по индексу. n_live_tup / n_dead_tup: оценка количества "живых" и "мертвых" (удаленных/обновленных, но не вычищенных) строк.
3. Значения	Даты (timestamp), счетчики (integer — накопление с момента старта базы или сброса), оценки (integer — текущий срез).
4. Практиче применение	Диагностика устаревшей статистики (почему планировщик строит плохие планы), выявление таблиц, требующих VACUUM (много n_dead_tup), и проверка эффективности индексов (соотношение idx_ scan к seq_scan).
5. Пример из ДЗ	В Практикуме 1 перед бенчмарком проверялся столбец n_mod_since_analyze, чтобы убедиться, что статистика таблицы ticket_flights актуальна (значение 0) и планировщик не ошибется при выполнении запросов.

Критерий	Описание
6. Ограничения	Показывает приблизительные оценки (n_live_tup), а не то количество строк (COUNT(*)). Счетчики сбрасываются падении сервера, если не настроено их сохранение.

2. pg_stat_activity

Критерий	Описание
1. Тип и схема	Системное представление (view), находится в схем pg_catalog.
2. Ключевые столбцы	state: текущее состояние сессии (active, idle, idle in transaction). wait_event_type / wait_event: тип и конкретное событие, которого ждет процесс (например, блокировка или чтение с диска). query: текст текущего запроса. query_start: время начала выполнения. backend_type: тип процесса (клиентский или системный фоновый).
3. Значения	Текст (text), метки времени (timestamp). Это строгий срез (snapshot) происходящего в базе дан ровно в момент выполнения запроса представлению.
4. Практическое применени	Мониторинг в реальном времени. Поиск "зависших" транзакций, выявление блокировок (deadlocks) и узких мест (latch contention, ожид I/O).
5. Пример из ДЗ	В Практикуме 4 (Hot Spot) представл опрашивалось в цикле для фиксации wait_events. Выявило LWLock|WALWrite при вставке BIGSERIAL (конкуренция за страницу) и полное их исчезнове при UUID.
6. Ограничения	Это моментальный снимок (срез), а не история. О быстрые запросы или кратковременные блокиров (как в Практикуме 6 с work_mem = '1MB') могу успеть попасть в выборку при редком опросе.

3. pg_stat_statements

Критерий	Описание
1. Тип и схема	Внешнее расширение (extension) и одноименное представление. Требует загрузки через shared_preload _libraries.
2. Ключевые стол	queryid: уникальный хэш запроса. calls: число выполнений. total_exec_time / mean_exec_time: суммарное и среднее время выполнения. shared_blks _hit / shared_blks_read: попадания в кэш и физические чтения с диска. temp_blks_written: запись временных блоков на диск. rows: суммарное количество возвращенных строк.
3. Значения	Счетчики и время (integer, float). Это долгосрочное накопл метрик по каждому нормализованному запросу.
4. Практиче применение	Профилирование нагрузки. Поиск самых долгих и ресурсоемких запросов, анализ cache hit ratio, выявление запросов, которым не хватает work_mem (сброс на диск).
5. Пример из ДЗ	В Разделе 9 представление выявило запрос ORDER BY amount DESC, который имел самое большое avg_time_ms, низкий hit ratio (0.73%) и активно писал temp_blks_written из-за нехватки памяти.
6. Ограничения	Нормирует литералы (скрывает конкретные параметры WHERE id = ?), из-за чего нельзя проанализировать п роблему со skewed data (перекосом данных). Не собирает данные без изменения конфига сервера.

4. pg_class

Критерий	Описание
1. Тип и схема	Системный каталог (таблица), находится в схеме pg_ catalog.
2. Ключевые столбцы	relname: имя объекта. relkind: тип (r - таблица, i – индекс). relpages: размер в страницах (по 8 КБ). reltuples: примерное количество строк. В связке используются функции pg_relation_size() (размер только данных) и pg_total_relation_size() (данные + индексы + TOAST).

Критерий	Описание
3. Значения	Текст, целые и вещественные числа. Текущий физический состояния объектов.
4. Практиче применение	Оценка физического размера таблиц и индексов на диске поиск "раздутых" (bloated) объектов, оценка плотности данных.
5. Пример из ДЗ	В Практикумах 2 и 4 использовалось для демонстрации того, как индекс BRIN (24 kB) в тысячи раз меньше B-Tree (44 MB), и как B-Tree по UUID весит больше, чем по BIGSERIAL из-за фрагментации.
6. Ограничения	Поля relpages и reltuples обновляются только фоновым процессом autovacuum/analyze. Если таблица активно меняется, цифры в этих столбцах быстро устаревают.

5. pg_prewarm

Критерий	Описание
1. Тип и схема	Внешнее расширение (extension) и функция pg_prewarm().
2. Ключевые столбцы	Как функция принимает параметры: regclass таблицы/индекса), mode (prefetch, read, buffer). Возвращает количество блоков, успешно загруженных в кэш.
3. Значения	Целое число (integer) — количество страниц (срез результата выполнения).
4. Практиче применение	Принудительный "прогрев" кэша базы данных (shared_buffers) или кэша ОС перед проведением бенчмарков, либо при вводе реплики/нового сервера в эксплуатацию, чтобы избежать "холодного старта" и тормозов при первых запросах.
5. Пример из ДЗ	В Практикуме 1 применялась для таблицы ticket_flights нового индекса, чтобы последующие тесты скорости EXPLAIN ANALYZE не искажались медленным первичным чтением с жесткого диска.

Критерий	Описание
6. Ограничения	Бесконтрольное использование "вымывает" из оперативной памяти другие полезные данные. Режим prefetch работает асинхронно и зависит от кэша ОС, не гарантируя, что данные останутся в RAM.

6. pgstattuple / pgstatindex

Критерий	Описание
1. Тип и схема	Внешнее расширение (extension) и набор функций.
2. Ключевые столбцы	avg_leaf_density: средняя плотность листовых страниц (%). leaf_fragmentation: доля фрагментированных страниц (%). leaf_pages: количество листовых страниц. index_size: общий физический размер. free_percent: процент свободного места.
3. Значения	Проценты (float) и байты/штуки (integer). Текущий структуры индекса/таблицы.
4. Практиче применение	Глубокая диагностика физической структуры B-Tree индексов. Выявление необходимости REINDEX и настройка параметра fillfactor под конкретную нагрузку (UPDATE-heavy или хаотичные вставки).
5. Пример из ДЗ	В Практикуме 5 функции наглядно показали, как индекс с резервом fillfactor=70 удерживал плотность н уровне ~69.8%, занимая больше физических страниц (leaf_pages), чем стандартный fillfactor=90 (88.8%).
6. Ограничения	Функции выполняют полное сканирование (full scan) таблицы или индекса. Запуск на "живой" высоконагруженной продакшен-базе большого объема может вызвать серьезные просадки производительности.

7. pg_stat_bgwriter (или pg_stat_io)

Критерий	Описание
1. Тип и схема	Системное представление (view), находится в схеме pg_catalog.

Критерий	Описание
2. Ключевые столб	checkpoints_timed / checkpoints_req: контрольные точки по расписанию и по требованию (превышение WAL). buffers_backend: блоки, записанные клиентскими процессами (плохо). buffers_checkpoint: блоки, сброшенные фоновым checkpointer (хорошо). buffers_alloc: общее количество выделенных буферов.
3. Значения	Накопительные счетчики (integer) глобально по всему инстансу базы данных.
4. Практиче применение	Общий взгляд на здоровье подсистемы ввода-вывода (I/O). Помогает настроить параметры shared_buffers и max_wal_size. buffers_backend растет быстрее buffers_checkpoint, значит кэш слишком мал или контрольные точки редкие.
5. Пример из ДЗ	*Применяется глобально для оценки того, справляется ли подсистема дисковой записи с потоком изменений.*
6. Ограничения	Статистика агрегируется по всему кластеру в целом. Невозможно понять, какой именно запрос или таблица генерирует аномальную дисковую нагрузку.

8. pg_stat_database

Критерий	Описание
1. Тип и схема	Системное представление (view), находится в схеме pg_catalog.
2. Ключевые столб	blks_hit / blks_read: суммарные чтения из памяти и с диска по всей БД. tup_returned / tup_fetched: прочитанные строки против фактически возвращенных. xact_commit / xact_rollback: количество успешных и отмененных транзакций.
3. Значения	Накопительные счетчики (integer) в разрезе конкретных баз данных.
4. Практиче применение	Общая картина здоровья базы. Быстрый расчет глобального cache hit ratio. Поиск баз данных с аномально высоким числом откатов транзакций (xact_rollback), что может свидетельствовать о программных ошибках в приложении.
5. Пример из ДЗ	*Позволяет оценить базовый уровень нагрузки*

Критерий	Описание
	(TPS) и эффективность кэширования для всей БД demo до внедрения оптимизаций.
6. Ограничения	Предоставляет только вид "с высоты птичьего полета". Высокий blks_read покажет наличие проблемы с памятью в базе, но для поиска виновника потребуется спускаться на уровень pg_stat_statements.

Для диагностики производительности СУБД наиболее полезными инструментами оказались pg_stat_statements и pg_stat_activity. Первое расширение позволяет осуществлять долгосрочный мониторинг и находить ресурсоемкие запросы, которые методично нагружают CPU или дисковую подсистему (как в случае со сбросом сортировки на диск). Второе представление незаменимо для real-time анализа инцидентов, например, при исследовании причин latch contention (горячих точек) при вставках с BIGSERIAL.

При следующей диагностике аналогичного стенда порядок действий был бы таким:

1. Оценка глобального Cache Hit Ratio через pg_stat_database, чтобы понять, хватает ли инстансу оперативной памяти в целом.
2. Проверка топа медленных и I/O-интенсивных запросов в pg_stat_statements.
3. Анализ планов выполнения проблемных запросов (поиск Seq Scan и external merge Disk).
4. Проверка статистики pg_stat_user_tables и размеров индексов в pg_class, чтобы исключить влияние устаревшей статистики планировщика или раздутых (bloated) индексов.